

一、程序功能介绍

(一) 核心交易功能：实现商品发现与交易闭环

1. 商品浏览与搜索

平台提供多样化的商品发现途径。用户可以通过预设的分类浏览结构（如按商品类别、品牌等划分）系统地探索商品库。关键词搜索功能允许用户输入具体的商品名称、特性或描述词汇，快速定位相关商品。在浏览和搜索结果页，强大的筛选功能可供用户根据多种条件（例如价格范围、商品状态、地理位置等可用选项）进一步缩小范围，精准定位符合个人需求的商品。这三者结合，确保了用户高效找到目标商品。

2. 商品发布

卖家享有便捷的商品上架流程。该流程设计简洁直观，引导卖家逐步完成信息填写。发布功能支持多图上传，卖家可以添加多张图片以多角度、全方位展示商品细节。卖家能够设置详细的商品描述，包括商品规格、使用情况、瑕疵说明等关键信息，确保信息透明。核心功能包含价格设置，卖家可为商品设定明确的出售价格，这是促成交易的基础信息。

3. 收藏功能

用户可将浏览过程中感兴趣的、但可能暂不购买的商品添加至个人收藏夹。此功能的主要目的是方便用户后续查看这些商品，无需重复搜索，便于集中管理和比较。

4. 评价系统

在交易流程顺利完成后，系统会引导买卖双方互相进行评价。用户基于实际交易体验（如商品符合描述程度、沟通顺畅度、交易效率等）给出评价。这些评价公开可见，共同构成了平台的信用体系，为其他用户提供交易参考，促进诚信交易环境的建立。

(二) 社交与账户管理：连接用户，掌控信息

1. 个人账户

用户拥有独立的个人账户中心，用于管理个人信息（如头像、昵称、联系方式等基础资料）。账户中心是用户查看交易记录的核心入口，清晰展示其作为买家或卖家的所有历史订单和当前交易状态。对于卖家用户，账户中心提供发布商品管理功能，可以查看、上架、下架自己发布的所有商品信息。

2. 私信沟通

平台内置点对点的私信沟通渠道，允许买家与卖家在交易前后直接进行交流。

该功能支持用户发送文字消息进行沟通，同时也支持用户发送图片消息（例如发送商品细节图、凭证截图等），使沟通更直观高效，解答疑问，促成交易。

3. 编辑功能

平台赋予用户灵活修改信息的权限。卖家可以随时编辑已发布的商品信息（包括图片、描述、价格等），以更新商品状态或修正信息。用户也可以随时编辑个人资料信息，确保个人展示信息的准确性和时效性。

（三）其他辅助功能：提升使用便捷与安全性

1. 收藏夹管理

用户可以对添加到收藏夹的商品进行整理和管理操作，包括查看收藏列表、移除不再感兴趣的商品等基础管理功能，帮助用户保持收藏夹的有效性和条理性。此外，平台发布的系统通知（如公告、规则更新提示等）也会在此显示，确保用户及时知晓平台动态。

2. 安全认证

平台提供基础的账户登录与注册功能，是用户访问和使用服务的前提。配套的安全设置选项（如修改密码、绑定手机 / 邮箱验证等）是核心组成部分，旨在帮助用户保护账户安全，防止未授权访问。

二、项目各模块与类设计细节

- 核心类设计：
- User 类（用户模型）
- ItemPost 类（商品模型）
- ChatRoom 类（聊天室）
- ChatMessage 类（消息模型）
- MainWindow 类（主窗口控制器）

1. User 类（用户模型）

核心功能定位：用户数据模型，负责用户信息管理、身份验证、评分系统等核心业务逻辑。

核心函数详解：

User 构造函数

创建新用户实例，接收用户名、密码、邮箱和认证等级参数。自动生成 UUID 格式的用户 ID，将密码转换为 SHA256 哈希值存储，记录用户注册时间，初始化用户的基本属性。

checkPassword()

验证用户登录时输入的密码是否正确。将用户输入的密码进行 SHA256 哈希处理，与数据库中存储的哈希值进行比较，返回布尔值表示验证结果。

addRating()

为指定用户添加评分和评价内容。接收评分者 ID、评分值（1-5 分）和评价内容，将评分数据存储在 QMap 中，支持重复评价时覆盖原有评分。

getAverageRating()

计算用户的平均评分。遍历所有评分数据，计算总分并除以评分人数，返回平均分，如果没有评分则返回 0.0。

getRatingLevel()

根据平均评分返回用户的评级等级。根据平均分范围返回对应的星级评价和文字描述，如 4.5 分以上为 "优秀"。

toJson()

将用户对象转换为 JSON 格式数据。序列化用户的所有属性，包括基本信息、评分数据、收藏商品、发布商品等，用于数据持久化。

fromJson()

从 JSON 数据创建用户对象。反序列化 JSON 数据，恢复用户的所有属性，包括评分、收藏等复杂数据结构。

2. ItemPost 类（商品模型）

核心功能定位：商品数据模型，管理商品信息、图片、标签、收藏等核心功能。

核心函数详解：

ItemPost 构造函数

创建商品实例，初始化商品的基本信息。接收商品标题、描述、价格、所有者、初始标签等参数，自动记录发布时间，建立与发布者的关联关系。

addToFavorites()

将商品添加到指定用户的收藏列表。检查用户是否已收藏该商品，如果未收藏则将用户 ID 添加到收藏列表中，避免重复收藏。

removeFromFavorites()

从指定用户的收藏列表中移除商品。在收藏列表中查找用户 ID，如果找到则从列表中删除，实现取消收藏功能。

isFavoritedBy()

检查指定用户是否已收藏该商品。在收藏列表中查找用户 ID，返回布尔值表示收藏状态，用于 UI 显示收藏按钮状态。

getFavoriteCount()

获取商品的收藏数量。返回收藏列表的大小，用于显示商品的受欢迎程度。

setMainImage()

设置商品的主图片。将指定的图片路径移动到图片列表的第一位，确保该图片作为商品的主要展示图片。

toJson()

将商品对象转换为 JSON 格式数据。序列化商品的所有属性，包括基本信息、图片路径、标签、收藏用户等，支持完整的数据持久化。

fromJson()

从 JSON 数据创建商品对象。反序列化 JSON 数据，恢复商品的所有属性，包括图片、标签、收藏等复杂数据结构。

3. ChatRoom 类（聊天室）

核心功能定位：即时通讯核心组件，管理双人聊天、消息历史、未读状态等。

核心函数详解：

ChatRoom 构造函数

创建聊天室实例，初始化聊天环境。接收两个用户和当前用户参数，设置聊天记录和未读状态的文件路径，加载历史消息和未读状态，建立聊天室的基本环境。

addMessage()

向聊天室添加新消息。创建新的 ChatMessage 对象，添加到消息历史中，在 UI 中显示消息，如果消息不是当前用户发送的则增加接收方的未读消息计数，保存消息历史并发出未读消息变化信号。

loadHistory()

从文件加载聊天历史记录。从 JSON 文件中读取历史消息，清空当前显示区域，重新显示所有历史消息，更新窗口标题显示未读消息数量。

saveHistory()

将聊天记录保存到文件。将当前的消息历史转换为 JSON 格式，写入到指定的文件中，如果保存失败则显示警告信息。

markAsRead()

将指定用户的消息标记为已读。将指定用户的未读消息计数清零，更新窗口标题，保存未读状态到文件，发出未读消息变化信号通知 UI 更新。

markAsUnread()

将指定用户的消息标记为未读。增加指定用户的未读消息计数，更新窗口标题，保存未读状态到文件，发出未读消息变化信号。

getUnreadCount()

获取指定用户的未读消息数量。根据用户身份返回对应的未读消息计数，用于 UI 显示和状态管理。

saveUnreadState () 和 loadUnreadState ()

保存和加载未读消息状态。将两个用户的未读消息计数保存到 JSON 文件中，或从文件中读取未读状态，确保程序重启后状态不丢失。

4. ChatMessage 类（消息模型）

核心功能定位：消息数据模型，负责消息的序列化和持久化存储。

核心函数详解：

ChatMessage 构造函数

创建消息实例。接收发送者、接收者和消息内容参数，自动记录消息的发送时间，初始化消息的基本属性。

toJson()

将消息对象转换为 JSON 格式。将消息的发送者、接收者、内容、发送时间等属性序列化为 JSON 对象，用于数据持久化。

fromJson()

从 JSON 数据创建消息对象。从 JSON 对象中提取消息属性，创建 ChatMessage 实例，恢复发送时间等数据。

saveMessages()

将消息集合保存到文件。接收消息向量和文件路径，将所有消息转换为 JSON 数组格式，写入到指定文件中，返回保存是否成功。

loadMessages()

从文件加载消息集合。从指定文件中读取 JSON 数据，解析为消息对象向量，如果文件不存在或读取失败则返回空向量。

5. MainWindow 类（主窗口控制器）

核心功能定位：应用程序的主控制器，负责 UI 管理、页面切换、业务逻辑协调。

核心函数详解：

createPages()

创建应用程序的所有页面。清空现有页面，创建个人主页、商品主页、私信页面三个主要页面，添加到页面容器中，建立完整的 UI 结构。

switchPage()

切换当前显示的页面。接收页面索引参数，验证索引有效性，切换到指定页面，在状态栏显示切换提示信息。

createChatRoom()

创建新的聊天室。检查是否已存在与目标用户的聊天室，如果存在则显示现有聊天室，否则创建新的聊天室，添加到活跃聊天室列表，连接信号槽，显示聊天窗口。

load_chat()

加载用户的聊天历史。遍历所有用户，检查是否存在聊天记录文件，为有聊天记录的用户创建聊天室，加载历史消息和未读状态，连接信号槽。

showPostEditor()

显示商品发布编辑器。创建商品发布对话框，包含图片上传、标题、价格、标签、描述等输入控件，处理用户输入，创建新的 ItemPost 对象，添加到商品列表并刷新界面。

refreshItemPosts()

刷新商品列表显示。保存商品数据到文件，重新创建所有页面，切换到商品主页，确保界面显示最新的商品信息。

showRatingDialog()

显示用户评分对话框。检查是否已评价过目标用户，如果已评价则询问是否重新评价，创建评分对话框，处理用户评分和评价内容，更新目标用户的评分数据。

showUserRatings()

显示用户的评分历史。创建评分历史对话框，显示平均评分、评级等级、评价人数等统计信息，列出所有评价者的评分和评价内容。

updateUnreadMessageDisplay()

更新未读消息显示。计算所有聊天室的总未读消息数，如果当前在个人主页则更新未

读消息按钮的文本和样式，如果当前在私信页面则更新未读消息标签。

createSinglePost()

创建单个商品卡片。根据商品信息创建商品展示卡片，包含图片、标题、价格、描述、标签、作者信息、收藏按钮等，设置点击事件和样式。

showDetailPage()

显示商品详情页面。创建商品详情对话框，显示完整的商品信息，包括大图、详细信息、作者信息、操作按钮等，支持编辑、删除、收藏等操作。

模块间关系设计

数据流设计

1. 用户数据流：User 类 JSON 文件 MainWindow
2. 商品数据流：ItemPost 类 JSON 文件 MainWindow
3. 消息数据流：ChatMessage 类 JSON 文件 ChatRoom MainWindow

信号槽机制

- ChatRoom 的 unreadCountChanged 信号 MainWindow 的 updateUnreadMessageDisplay 槽
- 实现了 UI 的实时更新和状态同步

全局数据管理

- globalusers.h/cpp：管理全局用户列表和当前用户
- global_itempost.h/cpp：管理全局商品列表
- 提供统一的数据访问接口

三、小组成员分工情况

- 前端：张景瑜，王泽恺，朱正雍。
- 后端与整合：王泽恺，朱正雍。
- 视频准备与文档撰写：张景瑜

四、项目总结与反思

项目概述

本项目是一个基于 Qt 框架开发的社交电商平台，成功集成了用户管理、商品发布、即

时通讯、评分系统等核心功能。项目采用 C++ 语言开发，使用 Qt 6.8.9 框架，实现了完整的桌面应用程序。

项目功能

用户系统：实现了完整的用户注册、登录、个人信息管理功能

商品管理：支持商品搜索、发布、编辑、删除、收藏等核心电商功能

即时通讯：实现了双人聊天、消息历史、未读消息提醒功能

评分系统：用户间相互评价，支持评分统计和等级显示

UI 界面：现代化的多页面设计，良好的用户体验

项目亮点

1. 架构设计：

模块化设计，每个核心类都有明确的职责边界，可扩展性良好：良好的接口设计，便于功能扩展

2. 用户体验

实时反馈：未读消息实时提醒，状态同步更新

直观操作：收藏、评分、聊天等功能操作简单直观

响应式设计：界面布局合理，交互流畅

3. 数据管理

完整性：支持复杂数据结构的序列化和反序列化

一致性：数据状态管理完善，避免数据不一致

持久化：程序重启后数据完整恢复

项目反思

不足：

- (1) 未使用数据库，数据管理依靠本地 json 格式文件管理相对简单
- (2) 缺乏网络功能，无法实现真正的多用户在线
- (3) 图片存储使用本地路径，不利于分布式部署

未来改进方向

1. 功能扩展

高级搜索：实现模糊搜索、分类筛选

推荐系统：基于用户行为的商品推荐

支付功能：集成支付接口，实现真实交易

2. 性能优化

缓存机制：引入内存缓存提高响应速度

异步处理：使用多线程处理耗时操作

资源优化：优化图片加载和内存使用

3. 用户体验

界面美化：进一步优化 UI 设计

操作简化：简化用户操作流程

个性化：支持用户个性化设置

总结

本项目成功实现了一个功能完整的社交电商平台，在技术实现、用户体验、代码质量等方面都达到了预期目标。通过项目开发，不仅掌握了 Qt 框架的使用，也深入理解了软件开发的完整流程。

虽然项目还有改进空间，但整体上是一个成功的实践项目。项目过程中遇到的问题和解决方案都成为了宝贵的学习经验，为今后的软件开发工作奠定了坚实的基础。